

Тестовое задание — Frontend Angular Developer

COLIZEUM · «Колесо удачи» для клубного приложения

Angular 17+

Анимация

~3-4 часа

Без бэкенда

Можно в портфолио

Задача

Реализовать SPA с экраном «Колесо удачи»: круглое колесо на 12 секторов, кнопка вращения, анимация поворота, модалка с результатом, лимит попыток, история.

1 Колесо удачи

- Круглое колесо на **12 секторов** равной величины (по 30° каждый)
- Каждый сектор подписан названием приза и/или иконкой
- Цвета секторов чередуются (минимум 2-3 цвета, чтобы визуально различались)
- Сверху колеса — неподвижная стрелка-указатель
- Реализация — на твой выбор: **SVG**, **CSS-conic-gradient**, либо **Canvas**. SVG предпочтительнее (легче подписи и адаптив)

2 Кнопка «Крутить» и анимация

- Кнопка запускает анимацию вращения
- Длительность анимации — **4 секунды**, плавное замедление (easing типа `cubic-bezier(0.17, 0.67, 0.16, 0.99)` или похожий ease-out)
- Колесо делает несколько полных оборотов и останавливается так, чтобы под стрелкой оказался выпавший сектор
- Во время вращения кнопка заблокирована
- После остановки — пауза ~500 мс, затем показывается модалка с результатом

3 Логика выпадения приза

- Призы и их веса (вероятности) загружаются **через HttpClient** из файла `assets/prizes.json` (см. список ниже)
- Алгоритм выбора — **взвешенный random**: чем больше вес, тем чаще выпадает
- Результат вычисляется **до** запуска анимации
- Анимация лишь визуализирует уже известный результат — финальный угол колеса детерминирован

⚠ КРИТИЧНЫЙ ПУНКТ

«Сначала случайно покрутить, потом посмотреть куда указывает стрелка» — это **неправильно**. Правильно: сначала выбрать приз алгоритмом, потом рассчитать целевой угол поворота колеса так, чтобы стрелка указала на нужный сектор, и запустить анимацию до этого угла. Так делают все настоящие игровые механики — иначе нельзя гарантировать честные вероятности.

4 Модалка с результатом

- Если выпал приз — показываем название, иконку, поздравление, кнопку «Забрать»
- Если выпало «Не повезло» — показываем сообщение «Попробуй завтра» и кнопку «Закрыть»
- Модалка с базовой анимацией появления (fade-in или scale)
- Можно закрыть по клику на оверлей или по Esc

5 Лимит попыток

- **3 попытки в день**, лимит хранится в `localStorage` вместе с датой
- В полночь (по локальному времени) лимит сбрасывается
- На экране показывается счётчик: «Осталось попыток сегодня: 2»
- Когда попытки закончились — кнопка заблокирована, надпись «Заходи завтра»

6 История попыток

- Внизу или сбоку — список последних **10 попыток**: дата, время, результат
- История хранится в `localStorage`
- Можно очистить кнопкой «Очистить историю»

Список призов (assets/prizes.json)

Используй ровно этот список — всего 12 секторов, нумерация по часовой стрелке от 12 часов.

#	НАЗВАНИЕ	ТИП	ВЕС
1	1 час игры на Standard ПК	приз	15
2	Не повезло	пусто	30
3	Скидка 30%	приз	15
4	Энергетик в подарок	приз	10
5	Не повезло	пусто	30
6	3 часа игры на Standard	приз	5
7	Скидка 50%	приз	8
8	Не повезло	пусто	30
9	200 ₽ на счёт	приз	10
10	1 час игры на VIP (Bootcamp)	приз	5
11	Не повезло	пусто	30
12	ДЖЕКПОТ — 5 часов на VIP	приз	1

Структура JSON

```
{
  "prizes": [
    { "id": 1, "label": "1 час Standard", "type": "prize", "weight": 15, "icon": "🎁" },
    { "id": 2, "label": "Не повезло", "type": "empty", "weight": 30, "icon": "—" },
    { "id": 3, "label": "Скидка 30%", "type": "prize", "weight": 15, "icon": "🎁" },
    ... всего 12 элементов
  ]
}
```

Веса не обязаны давать в сумме 100 — алгоритм должен сам нормализовать. Иконки можно заменить на свои SVG или эмодзи.

Технические требования

Angular: 17 или новее (приветствуется 20)

Компоненты: standalone, без NgModules

Шаблоны: новый control flow (@if, @for)

Состояние: Signals (попытки, текущий угол, флаг spinning, результат)

HTTP: HttpClient для загрузки prizes.json

Асинхронность: RxJS где уместно (отложенное открытие модалки и т.п.)

Анимация: CSS transform + transition, либо Angular Animations API

Хранилище: localStorage для лимита и истории

Типизация: strict: true, без **any**

Подписки: takeUntilDestroyed / async-пайп

Стили: SCSS, mobile-first, адаптив 375–1440px

DI: функция inject() в сервисах

Желаемая структура файлов

```
src/  
  app/  
    wheel/  
      wheel.component.ts      ← главный компонент колеса  
      wheel.component.html  
      wheel.component.scss  
      wheel-segment.component.ts ← один сектор (опционально)  
    result-modal/  
      result-modal.component.ts  
    services/  
      prize.service.ts        ← загрузка призов, взвешенный random  
      attempts.service.ts     ← лимит попыток + история (localStorage)  
    models/  
      prize.model.ts  
      attempt.model.ts  
  assets/  
    prizes.json
```

Критерии оценки

✓ Что оцениваем (в порядке важности)

1. **Детерминированность результата.** Сначала выбираем приз — потом крутим колесо до нужного угла. Если ты сделал наоборот — это сразу минус, даже если внешне красиво
2. **Корректность взвешенного random.** Веса должны реально влиять на вероятности (можно проверить простым тестом или счётчиком в консоли)
3. **Математика углов.** Колесо после анимации действительно показывает выпавший сектор, без расхождений
4. **Плавность анимации.** Минимум 60 fps, без рывков. Используется transform, а не изменение width/height/left/top
5. **Архитектура.** Логика колеса, призов, истории — в разных сервисах. Компонент не делает всё сам
6. **Реактивное состояние.** Signals для попыток, флага spinning, текущего угла. Шаблон обновляется автоматически
7. **Современный синтаксис Angular.** Standalone, новый control flow, inject(), функциональные guards (если нужны)
8. **Управление подписками.** Нет утечек памяти. Лучше — async-пайп или takeUntilDestroyed
9. **Типизация.** Модели призов и попыток типизированы, без **any**
10. **Лимит и сброс в полночь.** Работает корректно, не сбрасывается при перезагрузке страницы
11. **Адаптив.** Работает на мобильном (375px), колесо не уезжает за экран
12. **README.** Понятные инструкции запуска и краткое описание ключевых решений

Х Что НЕ оцениваем

- **Дизайн.** Достаточно опрятной вёрстки без визуальных багов. Не нужно делать «как в Wildberries»
- **Юнит-тесты.** Писать не обязательно (но плюс, если будут на сервисе призов)
- **Авторизация.** Не нужна
- **Реальная отправка приза на сервер.** Достаточно `console.log(prize)` при нажатии «Забрать»
- **Идеальный звук и звуковые эффекты.** Только в бонусах
- **Изысканные шейдеры и трёхмерное колесо.** Двумерное колесо вполне достаточно

Ожидаемое время и распределение

Базовая инициализация проекта, prizes.json, сервис загрузки	~20 мин
Колесо: SVG-разметка с секторами и подписями	~40 мин
Анимация вращения + расчёт целевого угла	~40 мин
Взвешенный random и логика выбора приза	~20 мин
Модалка с результатом	~25 мин
Лимит попыток + история в localStorage	~30 мин
Адаптив + полировка	~25 мин
README и проверка	~20 мин

Итого ~3-4 часа чистого времени

Если сильно растягивается — напиши, обсудим. Не нужно полировать неделю — нам важнее живое инженерное решение, чем идеальный пиксель.

Бонусные пункты (по желанию)

Не обязательно, но если успеваешь — впечатлит

- **Конфетти при джекпоте** — через canvas-confetti или своё на canvas
- **Звук вращения** и звук выигрыша — через Web Audio API или просто Audio()
- **@ngrx/signals (Signal Store)** для управления попытками и историей
- **Тесты на сервис** взвешенного random — проверить, что распределение реально соответствует весам (по большому числу запусков)
- **@defer** для ленивой подгрузки модалки
- **Совместимость с Webview** — корректное поведение в Android WebView / WKWebView (отключение pull-to-refresh, viewport, safe-area)
- **Анимация деталей** — пульсация кнопки, hover-эффекты, плавное появление модалки
- **Сохранение позиции колеса между сессиями** — после перезагрузки колесо не сбрасывается на 0°
- **Краткий ADR** в README: что и почему выбрал именно так (SVG vs Canvas, Animations API vs CSS, как считал углы)

Формат сдачи

1. Публичный **GitHub-репозиторий** с кодом
2. **README.md** с инструкциями: `npm install` + `npm start`
3. В README — 2–3 абзаца про ключевые решения: как считал угол поворота, как делал взвешенный random, какой подход к анимации выбрал и почему
4. **Опционально:** деплой на Vercel / Netlify / StackBlitz — это плюс для скорости ревью

Что дальше

1. Присылаешь ссылку на репозиторий
2. Мы смотрим (2–3 рабочих дня), даём обратную связь по существу
3. Если решение нас устраивает — приглашаем на финальное обсуждение: разберём твой код вместе, обсудим подходы и архитектурные решения, поговорим про условия

Если есть вопросы по постановке — пиши сразу, не угадывай. Лучше потратить 5 минут на уточнение, чем 2 часа на не то.

Вопросы и сдача работы

Илья · Telegram: [@frenzycs](#) · email: kim@colizeum.ru